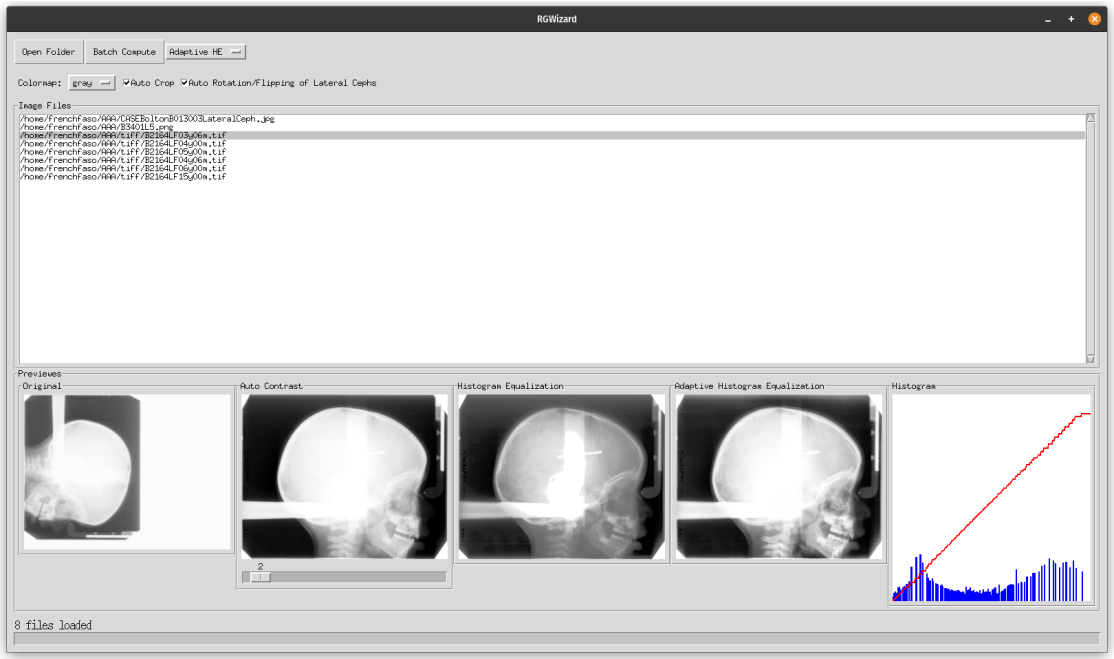
RGWizard

RGWizard is a cross-platform graphical user interface (GUI) application developed for the viewing and analysis of radiographic images, specifically tailored for the radiographs from the Bolton Brush Legacy Growth Study Collection. This collection predominantly features radiographs collected during the 1930s, though it extends until the 1980s.

The name RGWizard draws inspiration from the DICOM Modality (a key attribute of the DICOM standard), with "RG" representing Radiographic imaging (conventional film/screen), highlighting the application's focus on facilitating academic and research-oriented examination of radiographic scanned imagery, which could be decayed or otherwise in a poorer quality state compared to digitally acquired radiographs.

By employing various computational enhancement techniques, RGWizard serves as a specialized tool for researchers and academics working with this historically significant collection.



Overview

RGWizard integrates a suite of image processing algorithms to support radiographic research and analysis, particularly focusing on the unique needs and challenges associated with the Bolton Brush Legacy Growth Study Collection. It allows for the direct comparison between original and computationally enhanced radiographs images, aiding in the interpretation and study of radiographic data from this collection.

Features

- Original radiograph Image Viewing: Enables the examination of radiograph images in their unaltered state, providing a baseline for comparison with enhanced outcomes.

- Image Enhancement Techniques:
- Auto Contrast Adjustment: Implements automatic contrast adjustment to improve the visibility of radiograph image features.
- Histogram Equalization: Applies histogram equalization to redistribute the image's intensity, enhancing overall detail.
- Adaptive Histogram Equalization (CLAHE): Employs CLAHE for localized contrast enhancement, particularly beneficial for highlighting subtle nuances in radiograph images.
- Real-time Histogram Feedback: Incorporates a histogram display that offers immediate feedback on the intensity distribution changes resulting from the applied image enhancements, facilitating a deeper understanding of these processes.
- Automated Image Preprocessing:
- Auto Crop: Automatically identifies and removes extraneous white borders, focusing on the pertinent radiographic content.
- Auto Rotation and Flipping via AI: Utilizes a neural network model trained specifically for Lateral Cephalometric (Lateral Ceph) radiographs, automating the correction of image orientation through rotation and/or flipping. RGWizard is designed to be extensible with additional neural network models, and research is underway to develop models for automating other radiographic tasks such as the classification of different types of radiographs, rotation/flipping for other types of radiographs, and automatic segmentation of scans containing more than one radiograph.
- Batch Enhancement Application: Allows for the chosen enhancement technique, including automatic enhancements like auto crop and neural network model enhancements, to be applied in batch to all loaded radiographs. This feature streamlines the process of applying consistent enhancements across a collection of images, ensuring uniformity and saving significant time for researchers.

Usage

Installation

RGWizard can be packaged as a standalone executable, streamlining deployment and use without the need for a traditional installation process.

Prebuilt bundles, when available for a release, are published in the project's Releases section on GitHub (https://github.com/frenchfaso/RGW/releases). The current source tree can also be built locally with PyInstaller, as described below.

Windows users might require Microsoft Visual C++ redistributable package to ensure RGWizard runs properly. It can be downloaded from Microsoft's official site (https://visualstudio.microsoft.com/downloads/#microsoft-visual-c-redistributable-for-visual-studio-2022).

Building from Source

To compile RGWizard from the source code, one should first obtain the repository's contents. This can be achieved by cloning the repository or downloading the ZIP file containing the source code.

Following this, create a Python 3.12 environment. The current dependency set has been validated with Python 3.12 and PyTorch 2.12.

Using Conda

To create a Python environment with Conda, execute the following command in your terminal:

```
conda create -n rgw python=3.12 pip
conda activate rgw
```

This creates a new Conda environment named rgw and activates it.

Using Venv

For creating a Python environment with venv, the commands are as follows:

```
python3.12 -m venv RGWizard-env
source RGWizard-env/bin/activate  # On Windows, use `RGWizard-env\Scripts\activate`
```

This sequence creates and activates a new virtual environment named RGWizard-env.

Installing Dependencies

Once the environment is set up, install the required dependencies with:

```
python -m pip install -r requirements.txt
```

Building the Executable

Finalize the build process by generating a deployable bundle for your platform with PyInstaller. The models folder must be present; it is bundled into the executable by RGWizard.spec.

```
pyinstaller RGWizard.spec
```

This command compiles the application. After this step, you should find the generated bundle under the dist folder, ready for deployment. On macOS, PyInstaller also creates an app bundle. The build and dist folders are generated artifacts and can be removed and regenerated.